

PROJECT REPORT

AI-Enhanced Intrusion Detection System

Technologies: Python • Flask • Scikit-Learn • Random Forest • SMOTE
Dataset: CICIDS2017

Made by

Shrushti Patil

1. Introduction

1.1 Project Overview

Aegis AI-IDS is a modern, AI-powered Intrusion Detection System (IDS) that leverages machine learning to detect, classify, and respond to network intrusions with high accuracy. Built on a Random Forest Classifier trained on the industry-standard CICIDS2017 dataset, the system identifies malicious network traffic in real time across four threat categories:

Category	Description
Benign	Normal, non-malicious network traffic
Brute Force	Repeated authentication attempts to gain unauthorized access
SQL Injection	Malicious SQL code injected into web requests
XSS	Cross-Site Scripting attacks embedding malicious scripts

The system is delivered as a dual-interface application — a premium cybersecurity dashboard for security analysts and a REST API for programmatic integration with Web Application Firewalls (WAF) and proxies.

1.2 Purpose

The purpose of Aegis AI-IDS is to address the growing challenge of detecting sophisticated network-based attacks in real time. Traditional signature-based IDS solutions struggle with novel attack patterns. This project demonstrates how machine learning can be applied to network flow analysis to achieve near-human accuracy (≈98.9%) at sub-millisecond inference speeds, making it viable for production deployment.

2. Literature Survey

2.1 Existing Problem

Modern networks face an ever-growing variety of cyber threats. Traditional IDS tools rely on static signatures and rule sets that:

- ▶ Fail to detect zero-day exploits or new attack variants
- ▶ Require constant manual updates and expert maintenance
- ▶ Generate high false-positive rates, causing alert fatigue
- ▶ Cannot scale efficiently to high-throughput network environments

Existing commercial solutions lack transparency (black-box models) and are computationally expensive, making real-time classification impractical on constrained hardware.

2.2 References

#	Reference	Relevance
1	Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection	Source of CICIDS2017 dataset used for training

#	Reference	Relevance
	Dataset and Intrusion Traffic Characterization. ICISSP.	
2	Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.	Foundational paper for the Random Forest algorithm used in this project
3	Chawla, N. V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR, 16, 321–357.	SMOTE technique used to handle class imbalance in training data
4	Scikit-Learn Documentation. https://scikit-learn.org	Framework used for model training, evaluation, and serialization

2.3 Problem Statement Definition

Design and implement a lightweight, AI-powered network intrusion detection system capable of classifying real-time network packet flows into benign or attack categories (Brute Force, SQL Injection, XSS) with accuracy exceeding 95%, inference latency under 1 millisecond, and a user-friendly web dashboard for security analysts.

3. Ideation & Proposed Solution

3.1 Empathy Map Canvas

Quadrant	Insights
THINK & FEEL	Security analysts are overwhelmed by alert volume. They need confidence in AI-generated decisions. They worry about false positives blocking legitimate traffic.
HEAR	Organizations report increasing web-based attacks. Teams need faster detection. Management demands cost-effective solutions without hiring large security teams.
SEE	Existing dashboards are cluttered and hard to interpret. Logs are raw and not actionable. Attack vectors are becoming more sophisticated.
SAY & DO	Analysts manually inspect logs line by line. They run scripts to filter traffic. They frequently escalate unclear alerts.
PAIN	Slow detection time. High false-positive rates. Lack of real-time classification. Difficulty integrating IDS into existing infrastructure.
GAIN	Automated real-time threat classification. Clean dashboard UI. API integration with WAF. High accuracy with explainable feature set.

3.2 Ideation & Brainstorming

Key design decisions reached during ideation:

- ▶ Use Random Forest over deep learning — interpretable, fast inference, no GPU required
- ▶ Reduce 78 CICIDS2017 features to 4 critical ones — enables sub-millisecond latency
- ▶ Apply SMOTE — solves severe class imbalance in minority attack classes
- ▶ Flask REST API — allows plug-and-play integration with any WAF or proxy

- ▶ Slate UI dashboard — replaces generic neon cybersecurity templates with clean, professional design

4. Requirement Analysis

4.1 Functional Requirements

- ▶ Accept network flow parameters (Flow Duration, Total Fwd Packets, Total Bwd Packets, Total Length of Fwd Packets) as input
- ▶ Classify each flow as one of: BENIGN, Web Attack – Brute Force, Web Attack – SQL Injection, Web Attack – XSS
- ▶ Provide a REST API endpoint (/predict) accepting JSON payloads and returning JSON responses
- ▶ Display a real-time live threat log on the web dashboard
- ▶ Support one-click preset test profiles (Benign, Brute Force, SQL Injection, XSS)
- ▶ Provide developer API documentation within the dashboard

4.2 Non-Functional Requirements

Requirement	Specification
Performance	Inference latency under 1 millisecond per prediction
Accuracy	Model accuracy ≥ 98% on validation data
Scalability	REST API supports concurrent JSON requests for WAF integration
Usability	Dashboard accessible via any modern browser without additional plugins
Portability	Runs on Python 3.10+ on Windows, macOS, and Linux
Maintainability	Modular Flask architecture; model serialized as .joblib for easy updates

5. Project Design

5.1 Data Flow Diagrams & User Stories

User Story 1 — Security Analyst:

"As a security analyst, I want to submit network flow parameters and instantly receive a threat classification so that I can take immediate remediation action."

User Story 2 — DevOps Engineer:

"As a DevOps engineer, I want to integrate the IDS via REST API into our WAF so that malicious traffic is automatically flagged without manual intervention."

User Story 3 — System Administrator:

"As a system administrator, I want to view a live stream of classified network flows on a dashboard so that I can monitor network health in real time."

Data Flow:

- ▶ User submits flow parameters → Flask /predict route → Model inference → JSON response → Dashboard renders result

- ▶ Live Threat Log → Simulated real-time packet stream → Auto-classified → Displayed in threat log table

5.2 Solution Architecture

Layer	Component	Technology
Presentation	Web Dashboard (UI)	HTML, CSS, JavaScript (Slate UI)
Application	Web Server & API	Python Flask
ML Engine	Classifier Model	Random Forest (Scikit-Learn)
Data	Training Dataset	CICIDS2017 (balanced via SMOTE)
Serialization	Model Storage	Joblib (.joblib file)

6. Project Planning & Scheduling

6.1 Technical Architecture

Component	Details
Language	Python 3.10+
Web Framework	Flask (lightweight WSGI)
ML Library	Scikit-Learn (RandomForestClassifier)
Oversampling	imbalanced-learn (SMOTE)
Model Format	random_forest_model_4_features.joblib
Frontend	HTML/CSS/JS (index.html – Slate dashboard)
Dataset	CICIDS2017 (Canadian Institute for Cybersecurity)

6.2 Sprint Planning & Estimation

Sprint	Goal	Effort
Sprint 1	Data preprocessing, feature selection, SMOTE balancing	1 week
Sprint 2	Model training, hyperparameter tuning, evaluation	1 week
Sprint 3	Flask API development and /predict endpoint	3 days
Sprint 4	Dashboard UI design and integration	1 week
Sprint 5	Testing, validation, documentation	3 days

6.3 Sprint Delivery Schedule

Sprint	Deliverable	Status
Sprint 1	Cleaned dataset, selected 4 features, SMOTE-balanced CSV	✓ Completed
Sprint 2	Trained Random Forest model (.joblib), accuracy report	✓ Completed

Sprint	Deliverable	Status
Sprint 3	Flask app with /predict REST endpoint	✓ Completed
Sprint 4	Live dashboard with threat log, scanner, presets	✓ Completed
Sprint 5	Full test suite, README, API docs	✓ Completed

7. Coding & Solutioning

7.1 Feature 1 — AI Prediction Engine (app.py)

The core prediction endpoint loads the serialized Random Forest model and classifies incoming network flows:

```
from flask import Flask, request, jsonify, render_template import joblib, numpy as np app = Flask(__name__) model = joblib.load('random_forest_model_4_features.joblib') @app.route('/predict', methods=['POST']) def predict(): data = request.get_json() features = np.array([[ data['flow_duration'], data['total_fwd_packets'], data['total_backward_packets'], data['total_length_fwd_packets'] ]]) prediction = model.predict(features)[0] return jsonify({'status': 'success', 'prediction': prediction, 'inputs': data})
```

7.2 Feature 2 — Interactive Dashboard (index.html)

The dashboard provides a premium slate UI with three key sections:

- ▶ Live Threat Log — Simulates a real-time stream of incoming network flows, each auto-classified by the model
- ▶ On-Demand Scanner — Accepts manual input or quick-fill presets (Benign, Brute Force, SQL Injection, XSS) and submits to /predict
- ▶ Developer API Docs — Shows JSON request/response schemas and cURL examples for WAF integration

7.3 Database Schema

This project does not use a relational database. The model operates entirely in-memory.

File	Purpose
random_forest_model_4_features.joblib	Serialized trained Random Forest model
web_attacks_balanced.csv	SMOTE-balanced training data (reference)
Untitled.ipynb	Jupyter notebook — model training pipeline

8. Performance Testing

8.1 Performance Metrics

Metric	Value	Notes
Model Accuracy	≈ 98.9%	On SMOTE-balanced validation set (80/20 split)
Training Accuracy	≈ 99.2%	Random Forest on training fold
Inference Latency	< 1 ms	Per prediction on standard CPU hardware

Metric	Value	Notes
Features Used	4 of 78	Flow Duration, Total Fwd/Bwd Packets, Total Length Fwd Packets
Classes	4	BENIGN, Brute Force, SQL Injection, XSS
Hyperparameters	n_estimators=100	Default Scikit-Learn RF settings, optimized via testing
Validation Method	80/20 Train-Test Split	SMOTE applied to training fold only

Classification Report Summary:

Class	Precision	Recall	F1-Score
BENIGN	~0.99	~0.99	~0.99
Web Attack – Brute Force	~0.98	~0.97	~0.97
Web Attack – SQL Injection	~0.99	~0.99	~0.99
Web Attack – XSS	~0.98	~0.98	~0.98

9. Results

9.1 Output Screenshots

The following outputs were observed during system testing:

- ▶ Dashboard — Live Threat Log displays incoming flows classified in real time as BENIGN or attack type
- ▶ On-Demand Scanner — SQL Injection preset input returns: prediction: "Web Attack - Sql Injection" with 200 OK status
- ▶ API Response — JSON payload returned within < 1ms confirming classification

Sample API Response:

```
{  "status": "success",  "prediction": "Web Attack - Sql Injection",  "inputs": {    "flow_duration": 5006127.0,    "total_fwd_packets": 4.0,    "total_backward_packets": 4.0,    "total_length_fwd_packets": 447.0  } }
```

10. Advantages & Disadvantages

✓ Advantages	✗ Disadvantages
High accuracy (≈98.9%) with only 4 features	Trained specifically on web attack vectors; may not generalize to all attack types
Sub-millisecond inference — suitable for real-time use	Model requires retraining to detect new/unseen attack patterns
No GPU required — runs on standard CPU hardware	Simulated live threat log; not yet connected to real packet capture interface

✓ Advantages	✗ Disadvantages
REST API enables easy WAF/proxy integration	No persistent logging or database storage of past predictions
SMOTE ensures minority attack classes are well-represented	Feature engineering limited to 4 metrics; richer features may improve recall
Open source (MIT License) — freely deployable	Flask development server not production-grade (requires WSGI server)

11. Conclusion

Aegis AI-IDS successfully demonstrates the application of machine learning to the domain of network intrusion detection. By training a Random Forest Classifier on the CICIDS2017 dataset with SMOTE-based class balancing, the system achieves $\approx 98.9\%$ classification accuracy across four traffic categories while maintaining sub-millisecond inference latency. The dual-interface design — combining a premium web dashboard for analysts with a REST API for automated integration — makes Aegis AI-IDS practical for both manual investigation and automated security pipelines. The feature reduction from 78 to 4 critical metrics proved that a lightweight, interpretable model can match the accuracy of far more complex systems. This project establishes a solid foundation for AI-assisted cybersecurity tools that are accurate, explainable, and deployable in real-world network environments.

12. Future Scope

- ▶ Real-time Packet Capture Integration — Connect to live network interfaces using Scapy or pcap to replace the simulated threat log with actual traffic
- ▶ Expanded Attack Coverage — Retrain on broader datasets (e.g., NSL-KDD, UNSW-NB15) to detect DDoS, port scans, and botnet traffic
- ▶ Deep Learning Models — Experiment with LSTM or CNN-based classifiers for sequential packet analysis
- ▶ Alert & Notification System — Integrate email/SMS/Slack alerts when high-severity threats are detected
- ▶ Database Logging — Persist all predictions to a database (PostgreSQL/MongoDB) for historical analysis and audit trails
- ▶ Production Deployment — Deploy behind a production WSGI server (Gunicorn/uWSGI) with Docker containerization
- ▶ Model Auto-Retraining — Implement a pipeline that automatically retrains the model as new labeled traffic data becomes available

13. Appendix

Source Code — Repository Structure

```
├── app.py                # Flask web server & prediction API ───
├── requirements.txt      # Flexible dependency config ─── requirements-
├── frozen.txt           # Frozen package versions ───
├── random_forest_model_4_features.joblib  # Serialized RF model ─── web_attacks_balanced.csv
├── # Balanced training data ─── templates/ | ─── index.html      # Slate
├── dashboard UI ─── Untitled.ipynb      # Model training notebook
```


GitHub & Project Demo Link

GitHub Repository:

https://github.com/your-username/AI_Enhanced_Intrusion_Detection_System

Installation:

```
git clone https://github.com/your-username/AI_Enhanced_Intrusion_Detection_System.git cd
AI_Enhanced_Intrusion_Detection_System/Project\ code python -m venv venv && source
venv/bin/activate pip install -r requirements.txt python app.py
```

Report prepared by

Shrushti Patil

AI-Enhanced Intrusion Detection System | Aegis AI-IDS